

8 Counting Sort Algorithm

- We have talked about many algorithms so far
 - Insertion Sort** Worst: $\Theta(n^2)$, Average: $\Theta(n^2)$
 - Merge Sort** Worst: $\Theta(n \lg n)$, Average: $\Theta(n \lg n)$
 - Quick Sort** Worst: $\Theta(n^2)$, Average: $\Theta(n \lg n)$
 - Heap Sort** Worst: $\Theta(n \lg n)$, Average: $\Theta(n \lg n)$
- Most of the basic operations have been so far is comparison.
- Merge sort is asymptotically optimal (in general).
- Any comparison based sorting is bounded $\Omega(n \lg n)$
- Is there any way to sort in linear time?
 - Counting Sort is a way to achieve this.

8.1 Counting Sort

We want to sort integers. $[0 \dots k]$, where k is max in the array.

1. Count the number of each distinct elements (frequency).
2. Determine the position of each element by placing it to the “counting index”.

Specifically,

1. Specifically, given array of size n , and the maximum integer in the array is k .
2. Create “counting array”, whose size is $k + 1$.
3. Fill the counting array with the frequency of each element.
4. Update the counting array accumulatively adding up frequencies
5. Place our numbers in the position of “accumulated numbers”.

8.1.1 Example

Given Array: [1 3 0 2 4 3 1]

Size: 7

Max: 4

Counting array (size of $k+1$): [1 | 2 | 1 | 2 | 1]

Notice how the indexes are how many times the number appears in original array

Update: [1 3 4 6 7]
These are the updated positions of the array elements

Enumerated Steps

Output: [| | 1 | | |]

Now decrement updated array. ([1 2 4 6 7])

Output: [| | 1 | | 3 |] ([1 2 4 5 7])

Output: [| | 1 | | 3 | 4] ([1 2 4 5 6])

So on and so forth.

8.1.2 Psuedocode

Let A = input array, B = output array, n = size of array, k = maximum number.

```
countingSort(A, B, n, k)
  let C[0...k] be the counting array
  for i = 0 to k
    C[i] = 0
  for j = 1 to n
    C[A[j]] = C[A[j]] + 1
  for i = 1 to k
    C[i] = C[i] + C[i - 1]
  for j = n down to 1
    B[C[A[j]]] = A[j]
    C[A[j]] = C[A[j]] - 1
```

We can see that the complexity is $\Theta(n + k)$